

Chat API — Integration Guide

How to call POST /chat (the “Any Questions?” popup)

/chat cannot be used on its own: every question is answered against a session created by POST /summary, so the flow is always /summary → keep the session_id → /chat (up to 5 times).

Base URL — <https://agent.dtpay.digital/>.

1. Create a session with POST /summary

```
POST /summary
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
{
  "uid": 122,
  "start_date": "2026-07-28",
  "end_date": "2026-08-01",
  "cp_product_id": null
}
```

Field	Type	Required	Notes
uid	int	yes	must match the JWT's own uid row
start_date	date YYYY-MM-DD	yes	inclusive
end_date	date YYYY-MM-DD	yes	inclusive
cp_product_id	int null	no	omit for “everything this uid may see”

The range is inclusive of both ends and may not exceed MAX_DATE_RANGE_DAYS (default 7) — otherwise 400.

Response (trimmed):

```
{
  "session_id": "2fda4486-f5b2-4d6a-a649-ecdb0793a4e6",
  "date_range": { "start": "2026-07-28", "end": "2026-08-01" },
  "overall": {
    "total_resolved": 18432,
    "completed": 14120,
    "conversion_rate_pct": 76.6
  },
  "products": [
    {
      "product_id": 41,
      "product_name": "...",
      "summary": "...",
      "daily": [],
      "hourly": []
    }
  ],
  "generated_at": "2026-08-03T09:41:22.104512+00:00"
}
```

Store session_id. A session is created on every /summary call whether or not the user ever opens the chat, and it is the only handle /chat accepts. Its stored grounding data is exactly the overall + products payload above — the chat answers from that frozen snapshot, never from a fresh query.

2. Ask a question with POST /chat

```
POST /chat
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "uid": 122,
  "session_id": "2fda4486-f5b2-4d6a-a649-ecdb0793a4e6",
  "message": "Why did DRC dip on the 25th?"
}
```

Field	Type	Required	Notes
uid	int	yes	same uid that created the session, or an ADMIN
session_id	string (uuid)	yes	from the /summary response
message	string	yes	the user's question, free text

All three are required; a missing or wrongly-typed field is a 422, not a 400. The Authorization header is also required — a missing header is a 422 as well.

Authorization accepts either Bearer <jwt> (prefix match is case-insensitive) or the bare token.

Response — always exactly two fields:

```
{
  "session_id": "2fda4486-f5b2-4d6a-a649-ecdb0793a4e6",
  "answer": "DRC's conversion fell to 61.2% on the 25th, driven mainly by PAYER_LIMIT_REACHED on Vodacom, which account
```

answer is plain text, deliberately formatted for a popup with no markdown rendering: one continuous paragraph, no line breaks, no bullets or bold, typically 1–2 sentences. Render it as-is; don't run it through a markdown renderer.

curl

```
curl -X POST http://localhost:8000/chat \
-H "Authorization: Bearer $JWT" \
-H "Content-Type: application/json" \
-d '{"uid":122,"session_id":"2fda4486-f5b2-4d6a-a649-ecdb0793a4e6","message":"Which product had the worst day?"}'
```

Status codes

Status	When	Detail
200	answered	—
401	Invalid or expired credentials	Invalid or expired credentials
401	Has no dtpay_users	Unknown user
404	no session with that session_id	Session not found
410	session older than 24h	Session expired
403	session belongs to a different uid and request is not from ADMIN	This session does not belong to this user
429	5 questions already asked in this session	This session has reached its limit of 5 questions. Generate a new summary to keep asking questions
422	missing/invalid body field or missing Authorization header	Bad Request

Errors use standard `{"detail": "..."} body. Surface detail directly for 410/403/429 — those messages are written to be user-readable.`

A 410 or 429 means “start over”: call `/summary` again for the same date range to mint a fresh `session_id`, then keep chatting.

The fallback answer is a 200

If the Claude call fails or times out (`CLAUDE_TIMEOUT_SECONDS`, default 60s), `/chat` still returns 200 with:

“Sorry, I couldn't process that question just now - please try again.”

as the answer, rather than a 5xx — a failed answer shouldn't look like a server error to someone typing in a popup.